



DC – AI – Routing – Switching – Security – Wireless – Automation

PTX10001-36MR L2 Circuit Scale

An HPE Juniper Networking Techpost

Dmitry Shokarev

2022-10-06

HPE



Contents

1	PTX10001-36MR L2 Circuit Scale	3
1.1	Introduction	3
1.2	Configuration and Validation	4
1.3	Configuration Generation Script	6
1.4	Glossary	7

List of Figures

1	PTX10001 Internal Architecture	3
2	CCC Logical Interfaces, Circuits and Remote PEs.	4

1 PTX10001-36MR L2 Circuit Scale

Dmitry Shokarev - 10/06/2022

The PTX10001-36MR is well-known for its core, peering and DCI capabilities, but it's also a very performant L2 aggregation device. In this article, we will test and demonstrate L2 virtual circuits scale on Express4-based PTX routers.

1.1 Introduction

Each Express 4 ASIC (Codename: "BT") supports up to 16384 logical interfaces. In this test scenario we distribute logical interfaces between two PFEs of a PTX10001-36MR as shown in Figure 1:

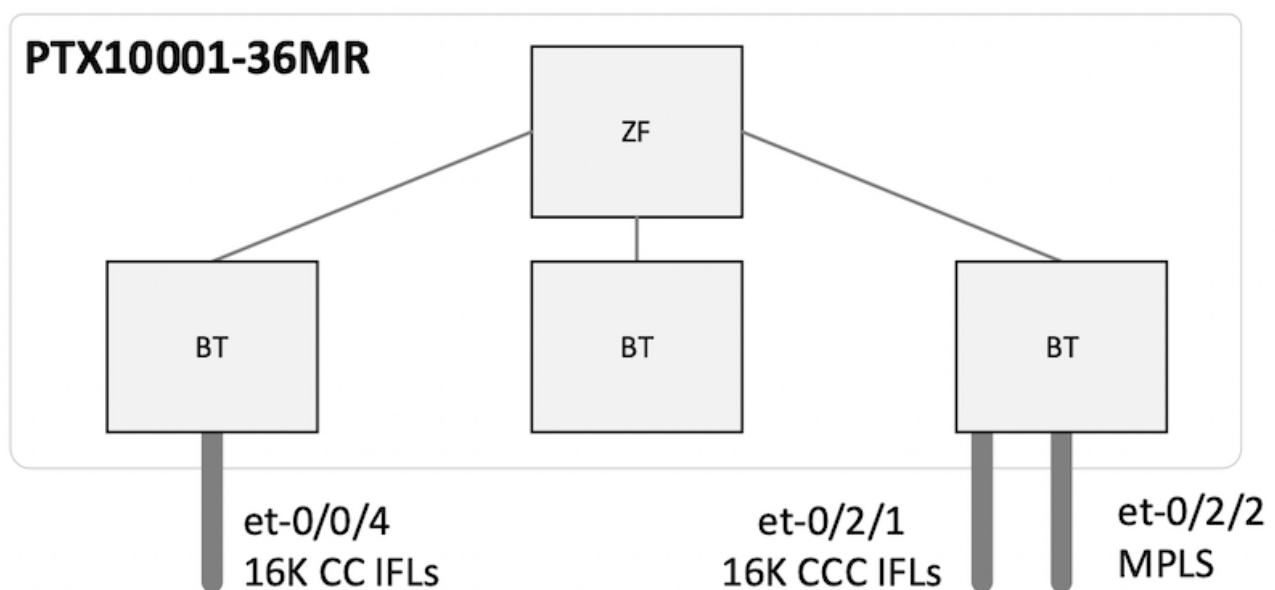


Figure 1: PTX10001 Internal Architecture

Figure 2 shows relationships between interfaces, circuits and remote edge devices. The setup has a trivial next-hop topology without any load balancing. Note that regular "et-" interfaces are used in this test. With Aggregated Ethernet in use the scale will reduce because, as of today, a separate logical interface is allocated for each member link (also known as "child" IFL).

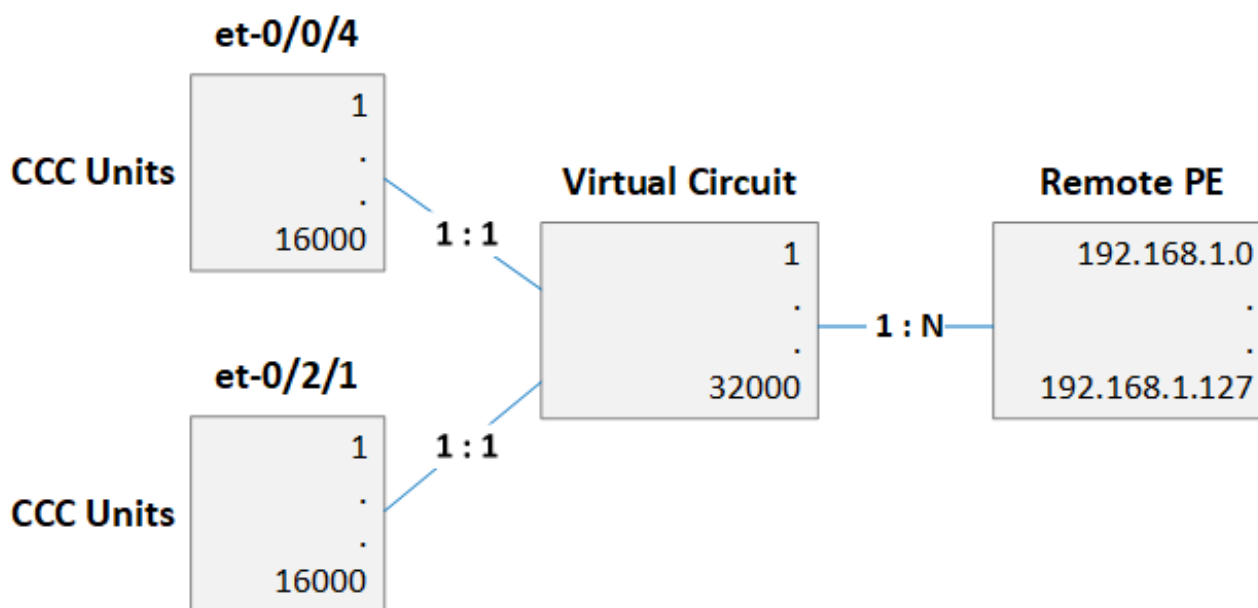


Figure 2: CCC Logical Interfaces, Circuits and Remote PEs.

1.2 Configuration and Validation

CCC interface configuration is shown below:

```

1 unit 1 {
2     encapsulation vlan-ccc;
3     vlan-tags outer 1001 inner 1;
4     input-vlan-map {
5         swap-swap;
6         vlan-id 1;
7         inner-vlan-id 100;
8     }
9     output-vlan-map swap-swap;
10 }
  
```

All the circuits are provisioned as static pseudo-wires (32000 total) and 128 static LSPs are provisioned to reach remote PEs.

All the l2 connections are “up” after commit:

```

1 regress@rtme-ptx10# run show l2circuit connections | match "rmt\s+Up" | count
2 Count: 32000 lines
  
```

PFE commands confirm that 32K MPLS and 32K CCC routes are present.

```

1 root@rtme-ptx10:pfe> show route summary
2
3 IPv4 Route Tables:
4 Index Routes Size(b) Prefixes Aggr Installed Comp(%)
5 -----
6 Default 35 3640 31 0 19 39
7 1 0 0 0 0 0
8 50 5 520 5 0 5 0
9 36738 5 520 5 0 5 0
10
11 MPLS Route Tables:
12 Index Routes Size(b) Prefixes Aggr Installed Comp(%)
13 -----
14 Default 32007 3328728 32007 0 32007 -
15
16 IPv6 Route Tables:
17 Index Routes Size(b) Prefixes Aggr Installed Comp(%)
18 -----
  
```

```

19 Default 7 728 7 0 7 0
20 1 0 0 0 0 0
21 50 5 520 5 0 5 0
22 36738 6 624 6 0 6 0
23
24 CLNP Route Tables:
25 Index Routes Size(b) Prefixes Aggr Installed Comp(%)
26 -----
27 Default 2 208 2 0 2 -
28 50 1 104 1 0 1 -
29
30 CCC Route Tables:
31 Index Routes Size(b) Prefixes Aggr Installed Comp(%)
32 -----
33 Default 32000 3328000 32000 0 32000 -

```

16K L2 circuits per PFE limit is imposed by the address space of the local L2 domain attribute of the ASIC (14 bits). The mapping of interfaces and VLANs to a logical interface is kept in the L2 domain table.

```

1 root@rtme-ptx10:pfe> show jexpr chash pfe 2
2 pfe : 2
3
4 ID PFE Name Size(cells) Used(cells) % MaxUsed(cells) % Note
5 ---
6 0 2 IGP-megidx 4096 0 0 0 0
7 1 2 EGP-megidx 4096 0 0 0 0
8 2 2 TunnelL2DomainHa 24576 0 0 0 1 entry=2/3/6 cells
9 3 2 TunnelL2DomainHa 24576 0 0 0 1 entry=2/3/6 cells
10 4 2 EPP_MAPID 196608 16120 8 16120 8
11 5 2 L2Domain 32768 16012 48 16012 48
12 6 2 SLU-MY-MAC 3072 39 1 39 1
13 7 2 DLU-IDB 2519040 0 0 0 1 entry=1/2/4 cells
14 8 2 SA-Learning 32768 0 0 0 0
15
16 ID PFE Name Size(cells) Used(cells) % MaxUsed(cells) % Note
17 ---
18 0 255 DLU-IDB 2519040 131287 5 415189 16 1 entry=1/2/4 cells

```

Utilization of ingress (next-hop) memory is less than 20%.

```

1 root@rtme-ptx10:pfe> show jexpr jtm ingress-main-memory chip 2
2 Show JTM Ingress MM allocation:
3 Pfe Chip: 2
4 [Global Region]
5 Used words: 129948, max used: 129948, allocs: 3792767, frees: 3754556
6 Allocator 0: used words: 10825, max used: 10825, allocs: 316064, frees: 312877
7   total words: 59392, available: 81%
8 Allocator 1: used words: 10828, max used: 10828, allocs: 316064, frees: 312880
9   total words: 63488, available: 82%
10 Allocator 2: used words: 10826, max used: 10826, allocs: 316064, frees: 312880
11   total words: 63488, available: 82%
12 Allocator 3: used words: 10828, max used: 10828, allocs: 316064, frees: 312880
13   total words: 63488, available: 82%
14 Allocator 4: used words: 10832, max used: 10832, allocs: 316064, frees: 312880
15   total words: 63488, available: 82%
16 Allocator 5: used words: 10828, max used: 10828, allocs: 316064, frees: 312879
17   total words: 63488, available: 82%
18 Allocator 6: used words: 10826, max used: 10826, allocs: 316064, frees: 312880
19   total words: 65536, available: 83%
20 Allocator 7: used words: 10832, max used: 10832, allocs: 316064, frees: 312880
21   total words: 65536, available: 83%
22 Allocator 8: used words: 10832, max used: 10832, allocs: 316064, frees: 312880
23   total words: 65536, available: 83%
24 Allocator 9: used words: 10832, max used: 10832, allocs: 316064, frees: 312880
25   total words: 65536, available: 83%
26 Allocator 10: used words: 10834, max used: 10834, allocs: 316064, frees: 312880
27   total words: 65536, available: 83%
28 Allocator 11: used words: 10825, max used: 10825, allocs: 316063, frees: 312880
29   total words: 65536, available: 83%
30 [Private Region]
31 Used words: 454, max used: 454, allocs: 521, frees: 5
32 Allocator 0: used words: 454, max used: 454, allocs: 521, frees: 5
33   total words: 4096, available: 88%
34 [Load Balance]
35 Allocator 0: used words: 144, max used: 144, allocs: 7, frees: 1
36   overflow, used words: 0, max used: 0, allocs: 0, frees: 0
37   total words: 6144, available: 97%
38 [SLT]

```

```

39     Allocator 0: used words: 0, max used: 0, allocs: 0, frees: 0
40         total words: 32768, available: 100%
41 [RPL]
42     Allocator 0: used words: 0, max used: 0, allocs: 0, frees: 0
43         total words: 2048, available: 100%
44 [REF]
45     Allocator 0: used words: 8, max used: 8, allocs: 1, frees: 0
46         total words: 8192, available: 99%

```

Egress memory is used for encapsulation structures, such as output VLAN IDs, forwarding MAC addresses and MPLS labels. 4 partitions out of 12 are in use and utilization of these 4 is less than 70%. Overall utilization is less than 30%.

```

1 root@rtme-ptx10:pfe> show jexpr jtm egress-memory summary chip 2
2 Show JTM Egress Memory Summary:
3 Pfe Chip: 2
4 Max Total Encap Memory Used (across all partitions in words) : 86155
5 Current Total Encap Memory Used (across all partitions in words) : 86155
6
7 PartitionID CurrType Total(words) Used(words) % MaxUsed(words) %
8 -----
9
10
11 0 MacTbl 32768 0 0 0 0
12
13 1 UnusedPool 32768 0 0 0 0
14
15 2 UnusedPool 32768 0 0 0 0
16
17 3 UnusedPool 32768 0 0 0 0
18
19 4 Public 32768 18680 57 18680 57
20
21 5 Public 32768 30720 93 30720 93
22
23 6 UnusedPool 32768 0 0 0 0
24
25 7 Public 32768 30720 93 30720 93
26
27 8 Private 32768 6035 18 6035 18
28
29 9 UnusedPool 32768 0 0 0 0
30
31 10 Private 32768 0 0 0 0
32
33 11 Tunnel 4096 0 0 0 0

```

1.3 Configuration Generation Script

```

1 import sys
2 import ipaddress
3
4 left_ifd = 'et-0/0/4'
5 right_ifd = 'et-0/2/1'
6 mpls_ifd = 'et-0/2/2'
7 first_nbr = ipaddress.IPv4Address( "192.168.1.0")
8 ifl_count = 16000
9 nbr_count = 128
10
11 def generate_config():
12     print("interfaces {")
13     print("\t{:{s}} {{{".format(left_ifd))
14     print("\t\tflexible-vlan-tagging;")
15     print("\t\tencapsulation flexible-ethernet-services;")
16     for i in range(0, ifl_count):
17         ivlan_left = i % 4000 + 1
18         ovlan_left = i // 4000 + 1
19         print( "\t\t\ttunit {:{d}} {{{".format( i+1))
20         print( "\t\t\t\tencapsulation vlan-ccc;")
21         print( "\t\t\t\tvlan-tags outer {:{d}} inner {:{d}};".format(ovlan_left, ivlan_left))
22         print( "\t\t\t\tinput-vlan-map {")
23         print( "\t\t\t\t\tswap-swap;")
24         print( "\t\t\t\t\tvlan-id 1;")
25         print( "\t\t\t\t\tinner-vlan-id 100;")

```

```

26 print( "\t\t\t")
27 print( "\t\t\t\toutput-vlan-map swap-swap;")
28 print( "\t\t\t")
29 print("\t}")
30 print("\t{:s} {}".format(right_ifd))
31 print("\t\tflexible-vlan-tagging;")
32 print("\t\tencapsulation flexible-ethernet-services;")
33 for i in range(0, ifl_count):
34     ivlan_right = i % 4000 + 1
35     ovlan_right = i // 4000 + 1 + 1000
36     print( "\t\t\tunit {:d} {}".format( i+1))
37     print( "\t\t\t\tencapsulation vlan-ccc;")
38     print( "\t\t\t\tvlan-tags outer {:d} inner {:d};".format(ovlan_right, ivlan_right))
39     print( "\t\t\t\tinput-vlan-map {")
40     print( "\t\t\t\t\tswap-swap;")
41     print( "\t\t\t\t\tvlan-id 1;")
42     print( "\t\t\t\t\tinner-vlan-id 100;")
43     print( "\t\t\t\t)")
44     print( "\t\t\t\toutput-vlan-map swap-swap;")
45     print( "\t\t\t)")
46 print("\t}")
47 print("}")
48 print("protocols {")
49 print("\tl2circuit {")
50 last_nbr_id = -1
51 vc_id = 1
52 for ifd in (left_ifd,right_ifd):
53     for i in range(0, ifl_count):
54         nbr_id = i // nbr_count
55         if nbr_id!=last_nbr_id:
56             if( last_nbr_id >= 0):
57                 print("\t\t\t") #close nbr container
58                 print("\t\t\tneighbor {:s} {}".format( str(first_nbr+nbr_id)))
59                 print("\t\t\t\tinterface {:s}.{:d} {}".format(ifd,i+1))
60                 print("\t\t\t\t\tstatic {")
61                 print("\t\t\t\t\t\tincoming-label {:d};".format(1000000+vc_id))
62                 print("\t\t\t\t\t\toutgoing-label {:d};".format(1000000+vc_id))
63                 print("\t\t\t\t\t)")
64                 print("\t\t\t\t\tvirtual-circuit-id {:d};".format(vc_id))
65                 print("\t\t\t\t)")
66                 last_nbr_id = nbr_id
67                 vc_id += 1
68 if( last_nbr_id >= 0):
69     print("\t\t\t") #close nbr container
70 print("\t}")
71 print("\tmppls {")
72 for nbr_id in range(0,nbr_count):
73     print("\t\t\tstatic-label-switched-path p{:d} {}".format( nbr_id))
74     print("\t\t\t\tingress {")
75     print("\t\t\t\t\tnext-hop 192.168.0.2;")
76     print("\t\t\t\t\tto {:s};".format( str(first_nbr+nbr_id)))
77     print("\t\t\t\t\tpush {:d};".format( 1000000+nbr_id))
78     print("\t\t\t\t)")
79     print("\t\t\t)")
80 print("\t}")
81 print("")
82
83 # execute
84 if __name__ == "__main__":
85     generate_config()
86 else:
87     pass

```

1.4 Glossary

- CCC: Circuit Cross-Connect
- IFL: Logical Interface
- LSP: Label Switched Path
- PE: Provider Edge
- PFE: Packet Forwarding Engine



DC – AI – Routing – Switching – Security – Wireless – Automation

© Copyright 2026 Hewlett Packard Enterprise Development LP. The information contained herein is subject to change without notice. The only warranties for Hewlett Packard Enterprise products and services are set forth in the express warranty statements accompanying such products and services. Nothing herein should be construed as constituting an additional warranty.

Hewlett Packard Enterprise shall not be liable for technical or editorial errors or omissions contained herein.

Trademark acknowledgments, if needed. All third-party marks are the property of their respective owners.

HEWLETT PACKARD ENTERPRISE

HPE.com

